LAN Collab — technical report draft

Implementation snapshot: 10 November 2026. Team members: fill in before submission. This report describes the implementation and local evidence; it is not a claim that the team has completed the manual acceptance checks or instructor review.

System architecture

LAN Collab is a central client-server application for students and study groups on a LAN. Each client has one persistent verified TLS connection. Global/private chat and files travel through the server. UDP discovery locates a candidate endpoint; TLS authenticates it.

```
Alice CLI/TUI ---- TCP + verified TLS ----+
Bob CLI/TUI ------ TCP + verified TLS ----+--> Central server
Carol CLI/TUI ---- TCP + verified TLS ----+    - accounts and token hashes: SQLite
      |                                       - active sessions and presence
      +----- UDP discovery/heartbeat -------->- bounded chat/file relay queues
                                              - no persisted file/chat content
Client storage: local chat-history SQLite; verified downloaded files; login token.
```

The runtime uses Python socket, ssl, selectors, threading, json, struct, hashlib and sqlite3. There is no HTTP framework or WebSocket layer. cryptography is a development-only certificate generation dependency. The optional TUI uses curses; the portable CLI remains available.

The default server capacity is 32 connections. One accept loop and one UDP worker are joined by one reader and one writer per admitted TLS connection. Shared application dispatch and queue admission are serialized. Socket writes occur outside the registry lock. Exact session IDs distinguish a new login from an old connection using the same username.

Requirement-to-implementation map

1. Protocol design: docs/protocol-spec.md and message_schema.py define envelope, states and limits.
2. Connection acknowledgment: OS TCP handshake, verified TLS, then AUTH_REQUEST/AUTH_RESULT.
3. Concurrency: bounded connection admission; separate reader/writer workers and nonblocking queues.
4. Authentication: scrypt hashes, first-admin CLI, pending registration, admin approval and presence.
5. Text routing: global fan-out/private forwarding with correlated server queue acknowledgment.
6. Files: consent, 16-KiB chunks, receiver credit, size/order/SHA-256 verification and safe commit.
7. Resilience: absolute deadlines, replay rejection, queue overflow handling, takeover and cleanup.
8. Encryption: certificate-verified TLS protects client-server links; the server sees plaintext.
9. UI: portable CLI, optional full-screen TUI, online list, progress bar and local chat history.
10. Verification: codec/core tests plus real TLS production-server integration and local load tests.

Protocol specification

A TCP application frame is a four-byte unsigned big-endian body length followed by exactly that many UTF-8 JSON bytes. Body length must be 1–262144. TCP can split one frame across reads or combine frames, so the receiver buffers bytes and extracts complete frames. Length limits are checked before allocating the advertised body. JSON duplicate keys are rejected.

```
+----------------------+-----------------------------------------------+
| uint32 BE body length | UTF-8 JSON: version, type, request_id, payload |
+----------------------+-----------------------------------------------+

{"version":1,"type":"CHAT_PRIVATE","request_id":"request-7",
 "payload":{"to":"bob","text":"Hello"}}
```

Version is the integer 1. IDs contain 1–64 printable ASCII characters. The schema rejects extra fields, incorrect types and unsupported message types. Direct responses copy the request ID; forwarded events have fresh server IDs. Each client session retains up to 65,536 request IDs; replays cannot repeat actions. Matching client PONG responses are exempt from this request cap.

Authentication and presence messages include REGISTER_REQUEST/RESULT, AUTH_REQUEST/RESULT, TOKEN_AUTH_REQUEST, PENDING_USERS_REQUEST/RESULT, ACCOUNT_REVIEW_REQUEST/RESULT, USER_LIST, PRESENCE_UPDATE and LOGOUT. Chat uses CHAT_GLOBAL, CHAT_PRIVATE and CHAT_ACK. Connection control uses PING/PONG and ERROR. There is no application HELLO/HELLO_ACK exchange.

```
TCP connected --> TLS verified --> WAIT_AUTH --> AUTHENTICATED --> CLOSING --> CLOSED
                                      |               |
                                invalid credentials     +--> chat, file, PING/PONG
                                retry below limit
                                3 failures/deadline --> CLOSING
```

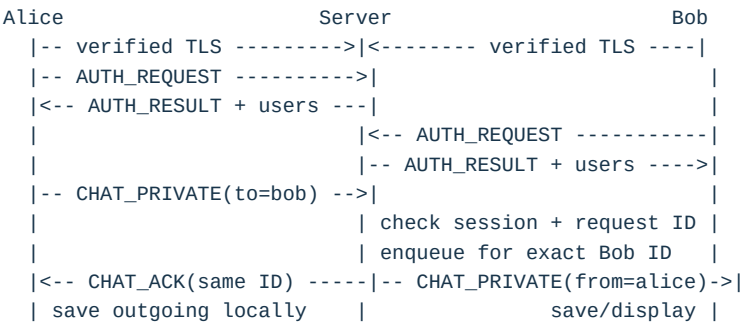
Passwords are verified against salted scrypt hashes. Network registration only creates pending member accounts; an existing admin approves or rejects them. Remember Me tokens expire after 30 days and can be revoked. The client stores the token; the server stores its SHA-256 hash. The distributed server-public.pem is a full self-signed certificate, not a bare public key. The server proves possession of server-secret.pem through TLS. No separate CA runs at handshake.

File messages are FILE_OFFER, ACCEPT, REJECT, CHUNK, ACK, END, RESULT and CANCEL (all prefixed FILE_). Transfer UUIDs bind exact sender/receiver sessions. The size cap is 50 MiB, chunk size 16 KiB, and sender window eight outstanding chunks. ACKs are cumulative at actual chunk boundaries after four written chunks or the final partial batch. FILE_END shares the chunk FIFO and does not wait for ACK drain. Success requires the receiver's final ACK and RESULT.

UDP messages are independent JSON datagrams, at most 1200 bytes. DISCOVER_REPLY echoes the request instance ID and advertises the TCP endpoint. HEARTBEAT_ACK echoes instance ID and sequence for RTT/loss measurement. These messages have no authentication/presence authority.

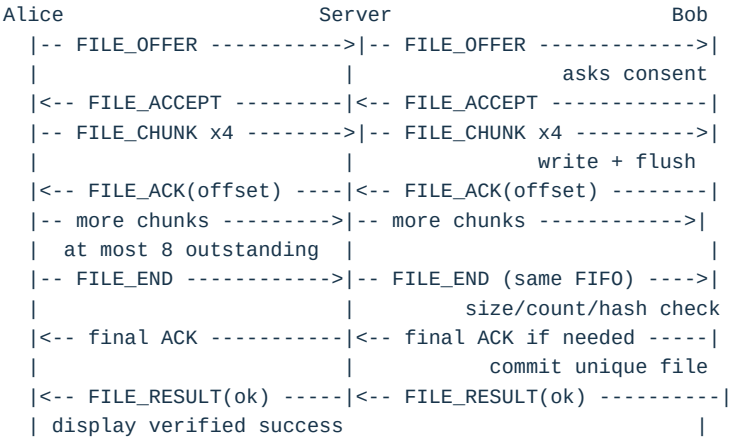
Message and file lifecycles

Private message



The ACK means accepted for queuing, not that Bob read the message. Bob must be online. A duplicate request ID gets ERROR without forwarding again. The authenticated session, not a sender-supplied field, determines the displayed sender identity.

File transfer



The final partial ACK may arrive before END. Empty files contain no CHUNK frames; END has zero chunks, followed by ACK offset zero and the verified result. Failures use CANCEL or a failed RESULT. The server does not independently hash file content or write it to disk. The receiver commits a non-overwriting filename only after verification; partial files are removed on cancellation/disconnect. At most one incoming transfer is admitted per receiver.

Challenges and solutions — review draft

This page summarizes engineering issues visible in the code and tests. The team should review and rewrite the reflection in its own words before submission.

TCP is a byte stream

A single send is not a packet boundary for the receiver. The codec therefore uses a fixed length prefix and exact body extraction. It handles partial and coalesced reads and rejects invalid sizes before allocation. Absolute frame deadlines prevent slow trickles from extending a partial frame indefinitely. The authentication deadline also caps partial-frame reads.

A slow client must not stall other users

Direct forwarding would perform socket writes in another client's request path. Each destination now has bounded control/file queues and one fair writer. Nonblocking admission bounds memory; control saturation closes that slow client and file saturation cancels the affected transfer. No global application lock is held during socket writes. TCP still has head-of-line blocking; queue priority cannot bypass bytes already written to the same TCP stream.

Completion is more than sending all bytes

A successful socket write only means bytes entered the transport. Receiver ACKs confirm written contiguous bytes and release application credit. Final success additionally requires matching size, chunk order/count and SHA-256, followed by safe destination commit. END shares the file FIFO so a prioritized control queue cannot send it before chunks. ACKs remain valid during verification, avoiding a deadlock between final acknowledgment and completion.

Login takeover and cleanup can race

A username is not a connection identity. Every connection receives a unique session ID. Login replacement updates mappings and cancels old transfers atomically. Cleanup checks the exact ID before deleting a username mapping, so a stale reader cannot mark the replacement offline. Presence snapshot and event admission run under the same application lock to retain order.

Replays and delayed errors must not repeat operations

The server stores bounded request-ID history and never evicts IDs to accept a replay. Direct responses copy IDs; files additionally correlate by transfer UUID. The older integration harness could misassociate a late transfer error with a newer offer and used a separate relay without production queue ordering. Tests now exercise the production server and correlate transfer errors with the original request instead of cancelling unrelated offers.

Authentication and local persistence have distinct lifetimes

TLS verification happens on every connection even with Remember Me. Credential prompts happen before the server's authentication timer begins. A failed network connection no longer erases a valid token; an explicit authentication rejection does. Local history records only received and server-acknowledged chat, isolated by trusted server identity and account. It is not encrypted at rest, and it is not an offline delivery service or an end-to-end encrypted protocol.

Verification and remaining acceptance work

The automated suite exercises framing/validation, TLS trust failures, transfer credit and state, client commands, accounts/tokens, local history and the actual threaded server. Production tests include 20 authenticated clients chatting during a 10 MiB transfer, ten concurrent transfer pairs, empty/small files, replay errors, deadline expiry, takeover, disconnects and stalled writers.

A separate reproducible workload is available:

```
python -m scripts.local_acceptance --output data/local-benchmark.json
python -m unittest discover -s tests -p 'test_*.py' -v
python -m ruff check .
```

The recorded docs/local-benchmark.json contains raw samples and environment details for 20 clients, 100 chat ACK timings, 100 TCP PONG timings and a verified 10 MiB file. These are local loopback measurements. The workload launches file transfer before concurrent chat requests; resource values describe the combined server/client process, not a server-only profile. Read the JSON for the exact run values rather than treating them as universal performance.

The following evidence must still be collected on the deployment machines: a two-host LAN run, UDP broadcast/fallback behavior, a Wireshark capture of TCP/TLS and UDP, server resource measurements, and manual terminal usability checks on supported platforms. See docs/acceptance.md for the exact workload command and environment notes.

The team must also supply member names, actual reviewer signoff, the final submission date and the instructor's encryption-rubric clarification. The software uses verified TLS; no E2EE, file resume, or offline delivery is claimed. There is no need for internet connectivity after the trusted application/certificate has been distributed.